The Y Language

A Formal Definition

the y compiler project

September 2026

Abstract

This document is the language definition of Y, a small statically-checked systems language that compiles directly to Linux x86-64 executables. It records the complete surface language, its type system, and its evaluation semantics — with particular precision for the mutating array operators (`+=`, `*=`), the increment/decrement and compound-assignment families, and their evaluated-exactly-once rules, so that these features are specified rather than discovered. Where the language is quirky, the quirk is written down; where a construct is reserved but unimplemented, it says so.

Contents

1 Overview **2**

2 Lexical structure **3**
2.1 Source text . . . 3
2.2 Comments . . . 3
2.3 Identifiers and keywords . . . 4
2.4 Literals . . . 4
2.5 Operators and punctuation . . . 4

3 Syntax **5**
3.1 Structure and modules . . . 5
3.2 Declarations . . . 5
3.3 Types . . . 6
3.4 Statements . . . 6
3.5 Expressions . . . 6

4 Types and static semantics **7**
4.1 Scalar types . . . 7
4.2 Arrays . . . 7
4.3 Records . . . 8
4.4 Maps . . . 8
4.5 Options and pointers . . . 8
4.6 Void . . . 8
4.7 Functions . . . 8
4.8 Annotations and inference . . . 9
4.9 Conversions . . . 9
4.10 Operator typing . . . 9
4.11 Compound assignment typing . . . 9
4.12 The poison law . . . 9

5 Expressions and evaluation **10**
5.1 Numeric semantics . . . 10
5.2 Logic . . . 10
5.3 Strings . . . 10
5.4 Assignment . . . 10
5.5 Increment, decrement and compound assignment . . . 10

6 Arrays **11**
6.1 Structure: length and capacity . . . 11
6.2 Operators . . . 12
6.3 Aliasing and visibility . . . 12
6.4 Fixed arrays . . . 13
6.5 Slices . . . 13
6.6 Byte strings . . . 13
6.7 Operators are the interface . . . 13

7 Records and maps **13**
7.1 The value model . . . 13
7.2 Records . . . 14
7.3 Maps . . . 14
7.4 Equality . . . 15
7.5 Copying . . . 15

8 Functions, closures, options, pointers **15**
8.1 Functions . . . 15
8.2 Tail calls . . . 15
8.3 Closures . . . 16
8.4 Options . . . 16
8.5 Void . . . 16
8.6 Pointers . . . 16

9 Builtins **17**
9.1 Sequences and text . . . 17
9.2 Conversions . . . 17
9.3 Binary data . . . 18
9.4 Options . . . 18
9.5 Input/output and the environment . . . 18
9.6 Pointers . . . 19

10 Errors and diagnostics **19**
10.1 Compile time . . . 19
10.2 Run time . . . 19
10.3 Exit status . . . 20
10.4 Reproducibility . . . 20

1 Overview

Y is a small imperative language with a static checker, designed so that a complete implementation of the language fits in one head: the compiler is roughly seventeen thousand lines of Y itself (`src/y/`, plus the runtime library it splices into programs) and compiles a program to a *static* x86-64 ELF executable using only its own assembler and ELF writer — no system assembler, linker, or C toolchain is involved at run time or at build time of user programs.

Implementations

Two implementations exist and are kept in exact step:

- **the reference compiler**, written in Y (`src/y/`), self-hosting and byte-reproducible;
- **the C copy**, a line-for-line port of the same compiler in C (`src/c/`), with its own value model (`src/c/yv.c`). The two must produce byte-identical executables for every input.

This document defines the language both implement. Implementation detail is deliberately excluded except where it *is* observable behavior (value sharing, integer wraparound, trap conditions).

Notation

Grammar productions are given in EBNF, with `typewriter` for terminals, *italic* for nonterminals, `[ ]` for option, `{ }` for repetition, `|` for alternation, and `( )` for grouping. Semantic rules are written as equational desugars or prose; the desugars are normative: an implementation may use any mechanism that produces the same observable behavior.

A taste

```
// values, references, and the mutating operators
let xs = [1, 2, 3];      // heap array: len 3
xs += 4;                 // in-place push:        [1, 2, 3, 4]
xs *= 0;                 // in-place prepend:     [0, 1, 2, 3, 4]
let ys = xs + [9];       // NEW array, xs untouched

// the target of a compound assignment runs exactly once
let a = [1, 2];
def idx() -> long { print("called"); 0 };
a[idx()] += 10;          // idx() runs ONE time

// records are reference values too
let r = { name: "y", nums: [1, 2] };
r.nums += 3;             // visible through every binding of r
```

2 Lexical structure

2.1 Source text

A program is a sequence of bytes, conventionally UTF-8. The lexer works on bytes; only ASCII is significant to the grammar. Both `"\n"` (LF) and `"\r\n"` (CRLF) end a line. Positions reported in diagnostics are 1-based line and column, the column counted in bytes.

2.2 Comments

```
// a line comment runs to the end of the line
/* a block comment
   does not nest: the first */ closes it */
```

Comments are whitespace: they separate tokens and nothing more.

2.3 Identifiers and keywords

`isalpha`(c) is true for `A`–`Z`, `a`–`z` and `_`; `isdigit`(c) for `0`–`9`. An identifier is

identifier ::= *isalpha* { *isalnum* }

so `_` is a word character: `is_digit` is a single identifier, not three tokens. The prefix `y_` is *reserved*: identifiers beginning with `y_` are used by the compiler's generated code and must not be written by hand.

The keywords are:

```
let const def if else while for foreach in return true false
```

2.4 Literals

Integers. Decimal, binary (`0b1010`) and hexadecimal (`0x1F`) radix literals are supported. A radix literal (`0x` / `0b`, either case) may separate digits with `_` in any position after the prefix: `0xFF_FF`, `0b1010_1010`. A *decimal* literal may not — `1_000_000` is a parse error, because the digit scan stops at the first `_` (and `_` is a word character, so the tail then lexes as an identifier). There is no octal form: `017` is decimal seventeen. All integer literals are *long* (64-bit).

Floats. A digit sequence with a decimal point and/or an exponent: `1.5`, `1.5e+3`, `2.0e-1`. A dot followed by a non-digit is *not* a fractional part — `a..b` lexes as `a`, `..`, `b`. Float literals are IEEE-754 binary64 (*float*).

Strings. Double-quoted byte strings:

escape	meaning
`\n \t \r`	LF, HT, CR
`"\" "\"" "\'"`	backslash, quote
`\xHH`	byte with hex value `HH`
`\c` (any other *c*)	*c*; the backslash is dropped

An escape is read as a single byte, and an unrecognised `\c` simply drops the backslash (`"q\q"` is `"qq"`), so a NUL is spelled `"\x00"` — which is not `"\0"` (that is the character `0`). A `str` is a length-prefixed byte string, so an escaped NUL inside one is ordinary data and `len` counts it (use `[byte]` arrays, Section 9.3, for bulk binary data).

Byte strings. `b"..."` and `b'...'` are *byte string literals*: a `[byte]` value spelled like a Python `bytes` literal. The escapes are the string escapes above, so binary data writes naturally: `b"\x7fELF"` is the four-byte ELF magic. Either quote spells the same type; single quotes exist only for byte strings.

Booleans and none. `true`, `false`. The empty option is spelled `none`, a predeclared name rather than a keyword (Section 4.5). There is no `null`: the word is an ordinary (undefined) identifier.

2.5 Operators and punctuation

kind	tokens
two-character	`== != <= >= && || :: .. -> => ++ - += -= *= /= %=`
single-character	`+ - * / % < > = ! & . ? |`
punctuation	`( ) { } [ ] , ; :`

Maximal munch. The lexer always takes the longest operator that matches, exactly like C:

- `x+++y` is `x ++ + y`, not `x + ++y`;
- `x-y` is `x- y` (a parse error), not `x - -y`; write `x - -y` when negation is meant;
- `/=` is one token; `//` starts a comment, and `/ == y` must be written with a space to divide;

Contextual keywords. `none`, `match`, `type` and `void` are ordinary identifiers to the lexer: `none` is a predeclared name (Section 4.5), `match` is the match-expression keyword only when an expression and `{` follow it, and `type` starts a type alias only when followed by `name =`. Each stays usable as an ordinary name elsewhere.

3 Syntax

3.1 Structure and modules

A *program* is a sequence of *statement*s executed in order. Source files may be combined with imports; an import splices the imported file's text in front of the importing file's own code, under a namespace alias:

```
import io from "./std/io.y";   // io.f(...) addresses defs of io.y
```

An `export { a, b }` clause, when present, lists what a file offers to importers. It is *by convention* the last statement of a file, but nothing enforces that: an import splices the imported file's text and drops every `export` line, so a clause anywhere in the file has the same effect. (A file's defs are spliced whole; namespaces are resolved by rewriting, not by the export list.) Paths resolve relative to the importing file's directory.

3.2 Declarations

declaration ::= `let` *name* [`:` *type*] `=` *expr* `;`
| `const` *name* [`:` *type*] `=` *expr* `;`
| `def` *name* `(` *params* `)` [`->` *type*] *block*
| `type` *name* `=` *type* `;` (alias)

params is a comma-separated list of *name* `:` *type*, each optionally followed by a literal default (`name = literal`, which types an unannotated parameter). Annotations are **strongly recommended** but not enforced: an unannotated parameter takes the unknown type, which poisons quietly (Section 4.12) and is treated as a machine word by the backend. Return types are optional (inferred from the body, Section 4). A `const` declaration currently behaves exactly like `let`; the keyword records intent and is reserved for future immutability checking.

Function literals may also appear bound or anonymous: `let f = def (x: long) -> long { x };`.

3.3 Types

type ::= `long` | `byte` | `bool` | `float` | `str` | `void`
| `[` *type* `]` (heap array)
| `[` *type* `;` *nat* `]` (fixed array)
| *type* `?` (option)
| `*` *type* (pointer)
| `(` [*type* {, *type*}] `)` `->` *type* (function)
| `{` [*name* `:` *type* {, *name* `:` *type*}] `}` (record shape)
| `{` *type* `=>` *type* `}` (map)

A record shape is structural; the shape `{ k: str }` is written to mean "a record with at least a field `k` of type *str*" and is the conventional shape-hole annotation for AST-like data. `nat` is a non-negative integer literal. The type name `int` is accepted as a synonym of `long` in annotations.

3.4 Statements

statement ::= *declaration*
| *expr* `;`
| *assignment* `;`
| `return` [*expr*] `;`
| *block*
| `if` `(` *expr* `)` *block* [`else` (*block* | *statement*)]
| `while` `(` *expr* `)` *block*
| `for` `(` *stmt* `;` *expr* `;` *stmt* `)` *block*
| `for` *name* `in` *expr* `..` *expr* *block*
| `foreach` `(` *name* `in` *expr* `)` *block*
| *block*

block is `{` *statement** [*expr*] `}`: a block is an *expression* whose value is its trailing expression (if any). An empty block `{}` is legal and void; a trailing `;` makes the block void. `foreach` iterates the elements of an array; the `for` range form iterates `lo..hi` with the low bound inclusive and the high bound exclusive.

3.5 Expressions

primary ::= *long* | *float* | *string* | `true` | `false` | *name*
| `(` *expr* `)` | *block*
| `[` [*expr* {, *expr*}] `]` | `[` *expr* `;` *nat* `]`
| `b` *string* | `c` *character* (byte string / byte)
| `{` [*name* `:` *expr* {, ...}] `}` `{` [*string* `:` *expr* {, ...}] `}`
| `def` `(` *params* `)` [`->` *type*] *block*

postfix ::= *primary* { `(` [*args*] `)` | `[` *expr* `]` | `[` *expr* `?:` *expr* `?` `]` | `.` *name* | `!` | `++` | `-` }
unary ::= [`!` | `-` | `*` | `&` | `++` | `-`] *postfix*

Postfix `e!` unwraps an option: it is exactly `unwrap(e)` (Section 8.4) and, like every postfix form, binds tighter than the prefix operators — `!x!` is `!(x!)`. The two-character token `!=` is always inequality.

Operator precedence

Binary operators are left-associative (`::` is the exception: it is right-associative, so `1 :: 2 :: a` is `1 :: (2 :: a)`); lower rows bind tighter:

level	operators	notes
1	`||`	short-circuit
2	`&&`	short-circuit
3	`== !=`	structural equality
4	`< > <= >=`	numeric, or lexicographic *str*
5	`::`	one-element cons (Section 6)
6	`+ -`	numeric, string/array `+`
7	`* / %`	numeric

Assignment forms

assignment ::= *place* `=` *expr*
| *place* *compound-op* *expr* `+= -= *= /= %=`
| *place* `++` | `++` *place* | *place* `--` | `--` *place*

A *place* is a name, an index expression `a[e]` (arrays and maps), a slice `a[lo:hi]`, or a field expression `o.f` (records), possibly nested: `grid[0][1]`, `rec.v[2]`, `m['k']! [i]` (a map read is *V?*, so unwrap it before indexing). Field access is symbolic and exclusive: `r.f` for records, `m['k']` for maps. Pointers add `*p = e`. Section 5.5 defines the exact evaluation of every form; all of them evaluate the place's subexpressions **exactly once**.

4 Types and static semantics

Checking is static and *fatal*: a type error is a compile error. The type system is small — scalars, arrays, records, maps, options, pointers, functions and `void` — and mostly structural.

4.1 Scalar types

type	domain	notes
`long`	64-bit signed integers	wraparound on overflow
`byte`	0 . . . 255	stored in one byte inside `[byte]`
`bool`	`true`, `false`	
`float`	IEEE-754 binary64	
`str`	immutable byte strings	length-prefixed; may hold NUL

4.2 Arrays

`[T]` is a heap array of *T*, `[T; N]` a fixed-length array of *T* with *N* a literal. The element type is part of the type and arrays are **homogeneous**: every element of an array has exactly the element type. Heterogeneous array literals are rejected:

```
let bad = [1, "two"];        // error: array element: expected long, got str
```

This applies recursively — `[[1], [2]]` has type `[[long]]`, and every operator that mixes arrays and elements (Section 6) enforces the same rule.

4.3 Records

A record literal `{ a: 1, b: "x" }` has the structural shape `{ a: long, b: str }`; field order is preserved and is part of the type. A field read is legal when the static shape has the field. The shape-hole form `{ k: str }` matches any record with at least those fields and is used where the precise shape is not known statically (it poisons quietly rather than erroring — see Section 4.12).

4.4 Maps

`{K => V}` is a dynamic map from hashable keys K to one value type V. Reading `m[k]` always yields `V?`; see Section 7.3.

K is a hashable key type (`str`, `long`, `byte`, `bool`, `float`), but a **map literal only accepts `str` keys** — the checker's inference for a literal fixes the key type to `str`, so `{ 1: 5 }` infers a shape rather than a map. Build a non-`str`-keyed map from an annotated empty literal and store into it (Section 7.3).

4.5 Options and pointers

`T?` is "a T or nothing". Its empty case is `none`; `some`/`unwrap`/`is_some` and the postfix unwrap `x!` (Section 8.4) manipulate it. There is no `null`: `none` is the only empty value, and it exists only at option types — a plain `str` or `long` can never be empty, so `s == none` on a `str` is a type error. On its own `none` has the type "option of anything" (printed `none`) and adopts the option type its context expects. A plain T does not widen to `T?`; write `some(v)`. `*T` is a raw machine pointer; `&e` yields one (Section 8.6).

4.6 Void

`void` is the type of an expression that computes no value: a loop, an `if` without `else`, a call to a void function (Section 8.5), a bare `return`. Void is not a value: it cannot be bound, passed, printed or returned as one.

Disagreeing `if` arms. When the two arms of an `if`/`else` have different types, the whole `if` is `void` — a statement. That is legal wherever a statement is, and an error wherever a value is needed:

```
if (ok) { 1 } else { "one" };            // fine: a statement
let v = if (ok) { 1 } else { 2 };        // long
let w = if (ok) { 1 } else { "one" };    // error: cannot bind a void value to '
    w'
```

4.7 Functions

`(T, T, ...) -> R`. Return types are inferred when omitted and checked when written. A parameter annotation is optional: without one the parameter takes the unknown type, which poisons quietly (Section 4.12) and is treated as a machine word by the backend — so annotate every parameter that carries a string, array, record or option. Function values are first-class (they may be bound, passed, stored and called); see Section 8.

4.8 Annotations and inference

- `def` parameter annotations are **recommended**; a missing one takes the unknown type and never cascades a second diagnostic (Section 4.12).
- Return types are inferred from the body when omitted; when written, every `return` and the body's trailing value must conform.
- `let` annotations are optional and checked; an annotated *array literal adopts* the annotated element type: `let b: [byte] = [104, 105];` packs those elements one byte each. Adoption is restricted to literals — assigning an existing `[long]` to a `[byte]` binding is a type error (the layouts differ).
- An empty array literal `[]` is legal unannotated: there is nothing to infer an element type from, so the element type is *unknown* and it poisons quietly (Section 4.12). Annotate it to pin the element type — `let b: [byte] = [];`. An unannotated `{}` is the empty record; under a map annotation it is the empty map: `let m: {str => long} = {};`.

4.9 Conversions

The implicit conversions are: `long` *to* `float` (the only numeric widening), `byte` *to* `long`, `byte` *to* `float`, and the narrowing `long` *to* `byte` at an *annotated* `byte` position (where the value is masked to eight bits when it is stored). A `str` also flows into a `*T` position, since a string value is a byte address. Apart from these, nothing converts: there is no cast syntax.

4.10 Operator typing

Arithmetic requires numbers; if either side is `float` the result is `float` and the other side widens. `+` additionally allows strings (concatenation, with auto-stringification of the non-string side) and arrays (Section 6); arrays win over strings when both appear. Comparisons yield `bool`; `<...>=` on two strings is lexicographic byte order. `&&`/`||` accept any values: a non-zero value is true and the result is `bool` (Section 5). Equality works on any two values of the same type; it is by content for scalars and strings, structural for arrays, and by identity for records and maps (Section 7.4).

4.11 Compound assignment typing

For a target of type T and value of type U:

T	operators	rule
numbers	`+= -= *= /= %=`	$U = T$ (or widening)
`str`	`+=` only	$U = str$
array	`+=`, `*=` only	Section 6

Anything else is a compile error, e.g. "`arrays support only '+=' and '*=' (in place), not '-='`". `++`/`--` require a numeric target.

4.12 The poison law

Missing information (a missing annotation, an unanalyzable expression) produces an *unknown* type that unifies with everything. The law this enables: **the root cause is reported exactly once** — an unknown-typed piece never generates follow-on errors at every enclosing use. Concretely, unknown unifies silently in element checks, call arguments and branches (so an unknown arm never makes an `if` void); the missing annotation itself is the single diagnostic.

5 Expressions and evaluation

Evaluation is strict and left-to-right; every operand is evaluated exactly once per execution of the enclosing expression. Side effects include calls, reads of places with side-effecting subexpressions, and the mutating array operators.

5.1 Numeric semantics

`long` arithmetic is two's complement modulo 2^{64}: overflow *wraps*, it does not trap. Division and remainder truncate toward zero (C semantics) and are subject to:

- division or remainder by zero is a runtime error (Section 10);
- `LONG_MIN / -1` evaluates to `LONG_MIN` (the quotient does not fit and would fault on x86) and `LONG_MIN % -1` evaluates to `0`. Implementations must special-case these two.

`float` arithmetic is IEEE-754 binary64. Mixed long/float operands widen the long side.

5.2 Logic

`&&` and `||` short-circuit: the right operand is evaluated only when the left does not decide the result. Operands are not required to be `bool` — any non-zero value is true, and the result of either operator is a `bool`.

5.3 Strings

Strings are immutable values: concatenation `a + b` produces a new string. If exactly one side of `+` is a string, the other side is stringified (Section 9.2) and concatenated. Comparisons `< <= > >=` on two strings compare bytes lexicographically; `==` compares contents.

5.4 Assignment

`p = e` evaluates `e` and stores it into the place `p`. Assignment is an expression yielding the assigned value. Section 7 defines what storing into arrays, records and variables means for aliasing.

5.5 Increment, decrement and compound assignment

The four increment/decrement forms and the five compound-assignment operators share one rule:

> **Exactly-once rule.** The target place's subexpressions are evaluated *exactly once*. Writing `a[f()] += e` evaluates `f()` one time, no matter how the place is read and written.

Formally, let a *place* be $p ::= x \mid a[i] \mid o.f$, possibly nested. Define $\mathrm{cap}(p)$, the capture of a place into fresh variables (every non-name subexpression runs once):

$$\begin{aligned} \mathrm{cap}(x) &= \epsilon \\ \mathrm{cap}(a[i]) &= \mathrm{cap}(a);\ \text{ if } a \text{ is not a name: } \texttt{let } \beta = a\texttt{; let } \iota = i\texttt{;} \\ \mathrm{cap}(o.f) &= \mathrm{cap}(o);\ \text{ if } o \text{ is not a name: } \texttt{let } \beta = o\texttt{;} \end{aligned}$$

with $\mathrm{read}(p)$ and $\mathrm{write}(p, v)$ the corresponding read and store through the captured names. Then the *numeric* forms desugar as:

$$
\begin{aligned}
\texttt{p++} &\equiv \text{cap}(p);\ \{\ \texttt{let t = read}(p);\ \texttt{write}(p, \texttt{t + 1});\ \texttt{t}\ \} \\
\texttt{++p} &\equiv \text{cap}(p);\ \{\ \texttt{let t = read}(p)\ \texttt{+ 1};\ \texttt{write}(p, \texttt{t});\ \texttt{t}\ \} \\
\texttt{p-} &\equiv \text{cap}(p);\ \{\ \texttt{let t = read}(p);\ \texttt{write}(p, \texttt{t - 1});\ \texttt{t}\ \} \\
\texttt{-p} &\equiv \text{cap}(p);\ \{\ \texttt{let t = read}(p)\ \texttt{- 1};\ \texttt{write}(p, \texttt{t});\ \texttt{t}\ \} \\
\texttt{p op= e} &\equiv \text{cap}(p);\ \{\ \texttt{let t = read}(p)\ \textit{op}\ e;\ \texttt{write}(p, \texttt{t});\ \texttt{t}\ \}
\end{aligned}
$$

for $\textit{op} \in \{+, -, \times, \div, \%\}$ on numbers. Each form is an expression: postfix yields the *old* value, prefix the *new* one, and `p op= e` the new value of the place.

Examples.

```
a[f()]++;          // { let y_t = f(); let v = a[y_t];
                   //   a[y_t] = v + 1; v }   -- f() runs once
rec.v[g()] += 1;   // g() once; the read and the write use the same element
x.y.z += e;        // x and x.y evaluated once
```

Arrays. On array targets `+=` and `*=` are *primitive in-place* operations — they do not desugar through `+` (which copies) — and are defined in Section 6. On string targets only `+=` is legal and concatenates. Every other combination is a type error (Section 4.11).

Maps and slices. A map store `m[k] = v` evaluates `m`, `k` and `v` once each; a slice assignment `a[lo:hi] = b` the same way (Section 6).

6 Arrays

Arrays are the language's one growable sequence. They are **reference values**: a binding names an array *object*, and mutating operators write into that object so every binding sees the change. Arrays are **homogeneous**: one element type, enforced everywhere.

6.1 Structure: length and capacity

An array object carries two independent quantities:

	meaning	invariant
`len`	number of live elements	`len` ≥ 0
`cap`	elements the storage can hold	`len` $\leq$ `cap`

Indexing is trapped: reading or writing `a[i]` with `i < 0` or `i >= len` is a runtime error ("index out of bounds"). `cap` is never observable through operators — only `len` is (`len(a)`).

Growth policy (amortized doubling).

$$
\begin{aligned}
\texttt{cap}_0 &= 0 \\
\texttt{cap}_{k+1} &= \max(8,\ 2 \cdot \texttt{cap}_k) \qquad \text{when } \texttt{len} \text{ reaches } \texttt{cap}
\end{aligned}
$$

That is: the array grows *only* when the capacity is reached; the first growth yields capacity 8 and every later growth doubles it $(8, 16, 32, \ldots)$. Appending n elements one at a time therefore costs $O(n)$ copies total, $O(1)$ amortized per element. The capacity never shrinks.

6.2 Operators

Let a, b be arrays of T, and x a value of type T. The complete operator semantics:

expression	result	effect on operands	value types
`a + b`	new array $= a +\!\!+ b$	none	$T = T$
`a + x`	new array $= a +\!\!+ [x]$	none	type of x is T
`x + a`	new array $= [x] +\!\!+ a$	none	type of x is T
`x :: a`	new array $= [x] +\!\!+ a$	none	x is ONE element
`a += b`	a	appends every element of b, in place	$T = T$
`a += x`	a	pushes x, in place	type of x is T
`a *= x`	a	prepends x, in place	type of x is T
`a = b`	b	aliases: both names name one array	

In prose, matching the design intent exactly:

- `arr1 + arr2` — make a new array which is `arr1` concatenated with `arr2`; the operands are untouched.
- `arr1 += arr2` — append `arr2` to the end of `arr1`, *in place*.
- `arr1 = arr2` — both point to the same array in memory.
- `arr1 + elem` — a new array which is `arr1` concatenated with `[elem]`; only allowed if the element types match.
- `elem + arr1` — a new array which is `[elem]` concatenated with `arr1`.
- `+=` means in-place push (of an element) or append (of an array); `+` always means a *new* array; `*=` means prepend the item in place.

The one-thing rule. Whether `+=` pushes or appends is decided by the *shape of the value*: an array value appends its elements; any other value pushes as one element. Consequently, to push an array *as one element* of an array-of-arrays, wrap it in a one-element array literal:

```
let rows = [[1], [2]];
rows += [[3]];        // appends the single row [3]
rows += [[4]];        // and the row [4] -- one element each
```

Homogeneity. Every mixing form requires matching element types; the checker rejects mismatches — including the append case, where a non-matching array would splice wrong-typed elements:

```
let rows = [[1], [2]];
rows += [3];   // error: '+=' appended elements: expected [long], got long
```

`a += a`. The right side is captured (its length snapshotted) before appending, so `a += a` doubles `a` with a copy of its original elements.

6.3 Aliasing and visibility

Assigning arrays aliases; mutating operators are visible through every alias:

```
let m1 = [1, 2];
let m2 = m1;      // ONE array, two names
m1 += 3;          // m2 sees it: len(m2) == 3
```

By contrast `+` never mutates: `let ys = xs + [9];` leaves `xs` at its old length.

6.4 Fixed arrays

`[T; N]` arrays are allocated with exactly `N` slots; their length is part of the type, so they reject growth: `+=` and `*=` on a fixed array is a compile error ("cannot grow fixed array `[long; 3]` with `'+='`"). Indexing is bounds-trapped like heap arrays. A fixed array may be sliced; the result is a heap `[T]`.

6.5 Slices

`a[lo:hi]` evaluates to a **new** heap `[T]` holding the elements in [`lo`, `hi`). Missing bounds default to `0` and `len`; bounds are clamped (no trap). All four spellings are legal: `a[:]`, `a[lo:]`, `a[:hi]`, `a[lo:hi]`.

6.6 Byte strings

A `[byte]` array *is* a binary string: `b"..."` literals produce them, `+` concatenates, slices cut, and printing one shows the Python-style repr — printable ASCII verbatim, quote and backslash escaped, `\n \r \t` as escapes, and every other byte as `\xHH`:

```
print(b"\bs x7fELF");      // b'\bs x7fELF'
let hdr = b"AB" + b"\bs x00";
print(hdr);               // b'AB\bs x00'
print(hdr[0:2]);          // b'AB'
```

`\std/struct.y` builds on this with pack/unpack helpers for file headers and binary layouts (`u32be`, `get_i16le`, `pack`, `unpack`, ...).

6.7 Operators are the interface

There are no array mutator functions: `+=` and `*=` *are* push and prepend. `len(a)` is the only builtin whose argument is an array *as such*; `join(xs, sep)` also takes an array but of any element type (each element is stringified), and `bytes`/`from_bytes`/ `write_bytes` take a `[byte]`.

7 Records and maps

Y has *two* key/value types, deliberately distinct:

	record (static)	**map** (dynamic)
type	`{ a: long, b: str }`	`{str => long}`
keys	identifiers, fixed at compile time	any hashable value at run time
values	may differ per field	one homogeneous type V
layout	C struct: fields at fixed offsets	growable key/value store
keying	`r.a`	`m['a']`

7.1 The value model

category	types	assignment
values	`long byte bool float str` options	copies the bits
references	arrays (`[T]`, `[T; N]`), records, maps	**aliases** the object

"Reference" means: every binding of the same object names one storage, and every in-place mutation (`+=`, `*=`, field assignment, map stores) is observable through all of them.

7.2 Records

A record literal `{ name: "y", n: 1 }` allocates one record object whose shape is the literal's fields *as written*; field order is preserved. At run time a record is exactly a C struct: field *i* is the word at `+8i` from the record address, so `r.a` compiles to a single load at a compile-time offset.

```
let r = { v: [7], n: 1 };
let s = r;              // aliases the record
r.v += 8;               // s.v sees it
s.n = 2;                // r.n sees it
```

The shape is fixed: writing a field the shape does not have is a *compile error* (there is no runtime field growth), and a field read requires the field to exist statically.

Record spread. `{a: 0, ...obj}` builds a record whose shape is the *union* of the two shapes — entirely statically; no dynamic shape is involved. Entries merge with last-wins values and first-appearance positions, so `obj` may overwrite `a` and a later explicit key may overwrite what `obj` spread:

```
let b = {x: 1, y: 2};
let c = {y: 9, z: 3};
print({a: 0, ...b});      // {a: 0, x: 1, y: 2}
print({...b, ...c});      // {x: 1, y: 9, z: 3}
print({...b, y: 100});    // {x: 1, y: 100}
```

7.3 Maps

A map type is `{K => V}` where *K* is a hashable key type (`str`, `long`, `byte`, `bool`, `float`) and *V* is one homogeneous value type. Map literals use quoted keys (identifier keys are record syntax):

```
let m = {"a": 1, "b": 2};       // {str => long}
let empty: {str => long} = {}; // an empty map needs the annotation
m["c"] = 3;                     // insert, or overwrite in place
print(m["a"]);                  // 1 -- reads yield V? (here: long?)
print(m["zz"]);                 // none
print(m["a"]! + 1);             // 2 -- x! unwraps
print(m);                       // {"a": 1, "b": 2, "c": 3}
```

Key types. Only `str` keys can be written in a map *literal*: the checker's literal inference fixes the key type to `str`, so `{ 1: 5 }` is rejected as a shape rather than accepted as a `{long => long}`. Every other hashable key type works when the map is built from an annotated empty literal and stored into — which is the only way to write one today:

```
let m: {long => long} = {};   // the annotation is load-bearing here
m[1] = 5;                     // insert
print(m[1]);                  // 5 -- a read is still V? (here: long?)
print(m[2]);                  // none
print(m);                     // {1: 5}
```

`byte`, `bool` and `float` keys behave the same way; see Section 4 for the type-level rule.
Keying is symbolic and exclusive: `m['k']` is the map form and is reserved for maps; `r.a` is the record form. Reading `m[k]` yields *V*`?` — `none` when the key is absent, even though the map's type says *V* — and `m[k] = v` stores in place (the store is visible through every binding of the map). A store to an existing key keeps its position; a new key is appended, so maps print

in insertion order. Duplicate keys in a literal collapse the same way (the last value wins, the first position stays). `len(m)` is the number of entries. The map grows as needed (amortized doubling, like arrays). Values are homogeneous: storing a `str` into a `{str => long}` is a compile error, as is a key of the wrong type.

7.4 Equality

`==` and `!=` on two **arrays** are **structural**: same length, pairwise-equal elements (with string elements compared by content). It is never a pointer comparison; `arr == []` is an emptiness test.

On two **records** or two **maps**, `==` compares *identity*: it is true exactly when both names refer to the same object. Two equal-looking literals are two objects, so `{x: 1} == {x: 1}` is `false`; compare fields yourself, or copy the record and compare the copy's fields.

7.5 Copying

There is no implicit deep copy. To duplicate an array use `+` with an empty right side (`a + []`) or slicing (`a[:]`); records can be rebuilt field by field. The language deliberately makes sharing the default and copying explicit.

8 Functions, closures, options, pointers

8.1 Functions

A `def` declares a function; parameters are typed, the return type is inferred or checked. Write recursion with `self` — that is the portable spelling, and it is what a function *literal* (which has no name) has to use. A *named* `def`'s own name is also in scope inside its body, so `count(n - 1, acc + 1)` and `self(n - 1, ...)` are the same call:

```
def fact(n: long) -> long {
    if (n <= 1) { 1 } else { n * self(n - 1) }
};
```

Functions are first-class values: they may be bound (`let f = g;`), stored in arrays and records, passed to other functions, and called through any of those (`f(1)`, `fs[0](1)`, `r.handler(1)`). Calling a non-function is a compile error.

8.2 Tail calls

A self-call in *tail position* — the body's trailing expression, a `return`'s value, or an if-arm's trailing expression — recycles the stack frame instead of growing it: the arguments are evaluated, the frame is torn down, and control re-enters the function. Tail recursion therefore runs in constant stack space:

```
def sum_to(n: long, acc: long) -> long {
    if (n == 0) { acc } else { self(n - 1, acc + n) }
};
```

Calls outside tail position (`self(x) + self(y)`, `let t = self(x)`, inside loops) are ordinary calls and stack as usual. Instrumented builds (`-prof`) keep plain calls so every profiled entry pairs with its exit.

8.3 Closures

A function literal captures the enclosing bindings it reads — and only those that are already in scope where the literal is written (a name bound by a `let` *after* the `def` is not in scope and is an undefined-variable error, exactly like reading a local before its declaration). Captured bindings are **shared**: a write inside the closure is visible outside, and a write outside is visible inside.

```
def counter() {
    let n = 0;
    def tick() -> long { n = n + 1; n };
    tick
};
let t = counter();
t(); t();
print(t());        // 3 -- the one shared n
```

Each *invocation* of the enclosing function creates fresh bindings, so closures created in different iterations see different cells ("`let` semantics"):

```
while (i < 3) {
    let x = i;
    fns += def() -> long { x };
    i = i + 1;
};
```

8.4 Options

`T?` is the type of "a T or nothing"; its empty case is `none`. `some(v)` boxes a value as present, `is_some(v)` tests, and `unwrap(v)` — or its postfix spelling `v!` — returns the payload and is a *runtime error* on `none` (it is a programmer error, loudly). `x == none` is the is-empty check; an option of an array is an optional reference, not a copied one.

```
let line = readline();          // str?
if (line == none) { print("eof"); } else { print(line! + "!"); };
```

8.5 Void

A `def` whose result is not a value is `void`. Write `-> void` to say so: the body's trailing value is then discarded, and `return` takes no value. An unannotated `def` whose only exits are bare `return;` statements (or whose body ends in a statement) infers `void`. A void call is a statement; using it as a value — binding it, printing it, passing it as an argument — is a compile error:

```
def log(msg: str) -> void { print("log: " + msg); };
log("hi");                  // fine: a statement
let x = log("hi");          // error: cannot bind a void value to 'x'
```

8.6 Pointers

`*T` is a raw address of one T; `*p = v` stores, `*p` reads. The address-of operator `&e` is deliberately narrow:

- `&x` — address of a local or parameter;
- `&a[i]` — address of an array element (any array expression: `&grid[0][i]` is legal);

- `&arr` on a whole array — *both* kinds, `[T]` and `[T; N]` — or `&` of any temporary, is a compile error ("cannot take the address of a whole array"). A binding names the array *header*, not its first element, so `&arr` would hand out the header; write `&arr[0]`, which works for either kind.

Pointers are for interfacing with the system-call level (buffers for `read`/`write`-style calls); ordinary code uses arrays and records.

9 Builtins

The following functions are available without declaration. "Trap" marks a runtime error (Section 10).

9.1 Sequences and text

`len(x)` Length of a string (bytes), array (elements) or map (entries) as `long`. Records are rejected (a record's field count is part of its type).

`range(n)` `[0, 1, ..., n-1]`.

`substr(s, i, n)`
Substring, C-clamped: `i < 0` becomes `0`, `i >= len` yields `""`, `n < 0` or overflow runs to the end.

`trim(s)` Strip leading/trailing space, tab, CR, LF.

`join(xs, sep)`
Concatenate `[str]` with a separator.

`split(s, sep)`
Split on every occurrence of a non-empty separator; returns `[str]` (empty for empty separator).

`index_of(s, sub)`
First index of `sub` in `s`, or `-1`.

`last_index_of(s, sub)`
Last index, or `-1`.

`ord(s)` First byte as `long`, or `-1` for empty.

`chr(n)` One-byte string of `n` (masked to 8 bits).

`byte_at(s, i)`
Byte at `i` as `long`; traps out of bounds. Cheap per-byte access (no copy).

9.2 Conversions

`tostring(v)`
String form: numbers as decimal (floats with six decimals), `true`/`false`, strings unchanged, options as their payload or `"none"`, arrays as `"[e0, e1, ...]"`, records as `"{k: v, ...}"`, maps as `"{"k": v, ...}"` (string keys quoted).

`tonum(s)` Parse a decimal integer (`0` on garbage).

`typeof(v)`
The static type as a printable value (`typeof(none)` is `"none"`).

9.3 Binary data

`bytes(s)` `[byte]` of the string's raw bytes.

`from_bytes(b)`
String from a `[byte]` array.

9.4 Options

`some(v)` A present option holding `v`.

`is_some(v)`
`true` unless `v` is `none`.

`unwrap(v)`
The payload; **traps** on `none`. The postfix form `v!` is the same operation.

9.5 Input/output and the environment

`print(x)` Print a value followed by a newline to stdout.

`puts(s)` Write a string to stdout, no newline added.

`eputs(s)` Same, to stderr (keeps `-time` output and program output apart).

`readline()`
One line from stdin without its newline, or `none` at end of input (`str?`).

`clear_eof()`
No-op: `readline()` issues a fresh read on every call, so end of input is never latched. Present so the same source runs on every compiler.

`read_file(path)`
File contents as `str?`: `none` when the file cannot be read.

`write_file(path, s)`
Write a string; `bool` success.

`write_bytes(path, b, mode)`
Write `[byte]` binary-safe and chmod (`mode == -1` skips chmod — any other value is passed to `fchmodat` verbatim, so `493` is `0755`); `bool` success.

`argv()` `[str]` of the program's arguments.

`exit(code)`
Terminate with an exit code.

`clock_us()`
Monotonic microseconds as `long`.

`heap_used()`
Bytes allocated so far.

`sleep(ms)`
Suspend the process (`void`).

`syscall(...)`
Raw Linux syscall wrapper; the first argument is the syscall number.

`exec(cmd)`
: Run a shell command, return its exit code. Removed from the binary entirely by `-no-exec` (then calling it is a compile error).

9.6 Pointers

`ptr_addr(p)`
: The address of a pointer as `long`.

Mutating and keying are symbolic, not functional: arrays use `+=` / `*=`, records use `r.f`, maps use `m['k']` and `m['k'] = v`. The legacy functions `push`, `prepend`, `map_get` and `map_set` do not exist.

10 Errors and diagnostics

10.1 Compile time

Static checking is fatal: a rejected program compiles to nothing. Diagnostics are one line each,

```
line:col:  error:  message
```

positioned in the user's source even when the compilation splices prelude/runtime code in front of it. The checker reports a root cause *once* (Section 4.12): a missing annotation or unanalyzable expression poisons quietly instead of cascading errors through every enclosing use. Typical families:

- name errors: `undefined variable 'x'`;
- shape errors: `record has no field 'f'`;
- operator typing: `'+=' appended elements:  expected [long], got long`, `arrays support only '+=' and '*='`, `'++' needs a number target`;
- call errors: `call takes 2 args, got 1`;
- non-value use: `cannot index long (only arrays and maps)`;
- void use: `cannot bind a void value to 'x'`.

10.2 Run time

The compiled program traps — writes a one-line message to stdout and exits with status 1 — on:

condition	message
array index outside `[0, len)`	`runtime error:  index out of bounds`
division or remainder by zero	`runtime error:  division by zero`
`unwrap(none)`, `none!`	`runtime error:  unwrap of none`
missing dynamic field/key	`runtime error:  record has no field 'k'`
slice assignment out of range	`runtime error:  slice assignment out of bounds`
match with no arm covering a value	`runtime error:  no match arm matched`
memory exhausted	`runtime error:  out of memory`
unbounded (non-tail) recursion	stack exhaustion: no limit, so the process dies on the guard page

These messages go to **stdout** (the trap stubs use `write(1, ...)` directly), so a program's real output stays recognizable; `eputs` is the way to send diagnostics to stderr.
Fixed arrays trap exactly like heap arrays; slices never trap (they clamp). Integer overflow is *not* an error (Section 5).

10.3 Exit status

A successfully run program exits with the code it chose (`exit(code)` or falling off the end, which is `0`); a trapped program exits with `1`. The compiler itself exits `0` on success, `1` on a rejected input.

10.4 Reproducibility

The compiler is deterministic: the same input produces byte-identical output on every run and in either implementation. This is checked continuously by comparing the two compilers' output binaries byte for byte, and by self-compilation (a compiler built by the compiler is identical to the one that built it).